from the final representation of that data, called the *view*. The controller is responsible for deciding whether the response should generate an HTML view, whether it should send the user to a new page, or whether it should return an error page.

One of the advantages of the model's being independent of the view is that it improves testability. UI code is classically hard to test, as it's dependent on the environment; anyone who has written UI tests simulating a user clicking buttons and typing in forms knows that it's typically fragile. By keeping the model independent of the view, you can ensure that the model stays easily testable, without any dependencies on UI constructs. As the model often contains your application's business logic, this is clearly a good thing!

The view can use the data passed to it by the controller to generate the appropriate HTML response. The view is responsible only for generating the final representation of the data; it's not involved in any of the business logic.

This is all there is to the MVC design pattern in relation to web applications. Much of the confusion related to MVC seems to stem from slightly different uses of the term for slightly different frameworks and types of applications. In section 13.4 I'll show how the ASP.NET Core framework uses the MVC pattern with Razor Pages, along with more examples of the pattern in action.

13.4 Applying the MVC design pattern to Razor Pages

In section 13.3 I discussed the MVC pattern as it's typically used in web applications; Razor Pages use this pattern. But ASP.NET Core also includes a *framework* called ASP.NET Core MVC. This framework (unsurprisingly) closely mirrors the MVC design pattern, using *controllers* containing *action*